

中国科学院大学

《计算机网络(研讨课)》实验报告

姓名 寇逸欣 学号 2023K8009922004 专业 计算机科学与技术
实验项目编号 06 实验名称 数据包队列实验

一、 实验内容

1. 根据 PPT 及自己的调研,了解 bufferbloat 以及 TCP-Incast 的成因。
2. 利用所给代码,复现 bufferbloat,根据 PPT 要求绘图并且解释。
3. 利用所给代码,用 RED,CoDel 以及 tail drop 解决 bufferbloat,根据 PPT 要求绘图并解释。

二、 bufferBloat 和 TCP-Incast 的成因

2.1 BufferBloat 问题的成因

BufferBloat 问题产生的根本原因在于过大的缓冲区设置。TCP 的拥塞控制机制依赖于丢包或者延迟的增加来降低发送速率,但是,由于过大的缓冲区,即使链路饱和缓冲区也不会立即丢包,导致发送方没有获取到这个信息,从而继续以高速度发送数据包,最终导致网络延迟增加和吞吐量下降。而又因为延迟的增加 TCP 没有及时感知,造成了恶性循环:链路饱和->缓冲区变满->延迟增加->TCP 反应迟缓,降低发送速率->队列排空->TCP 又开始加速发送。

2.2 TCP-Incast 问题的成因

TCP-Incast 的成因主要是由于多个发送方同时向一个接收方发送大量数据包,导致接收方的缓冲区迅速填满,从而引发丢包和重传。具体来说,当一个客户端发出请求时,可能会有多个服务器同时响应,导致大量数据包在短时间内涌入接收方的缓冲区。如果缓冲区容量不足以容纳这些数据包,就会发生丢包(几乎每个发送方都会丢包)。由于超时重传机制所需要的时间较长,会导致长时间的链路空闲,链路利用率低,吞吐量急剧下降。

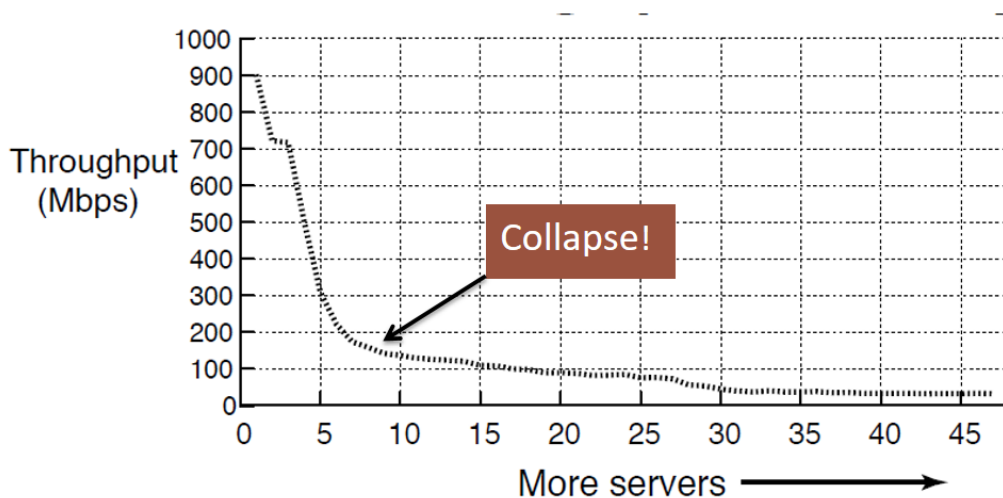


图 1: TCP-Incast 示意图

三、重现 bufferbloat 问题

先选取 100 长度的队列测量 h1 的拥塞窗口值 (cwnd)、r1-eth1 的队列长度 (qlen)、h1 与 h2 间的往返延迟 (rtt),结果如下图所示:

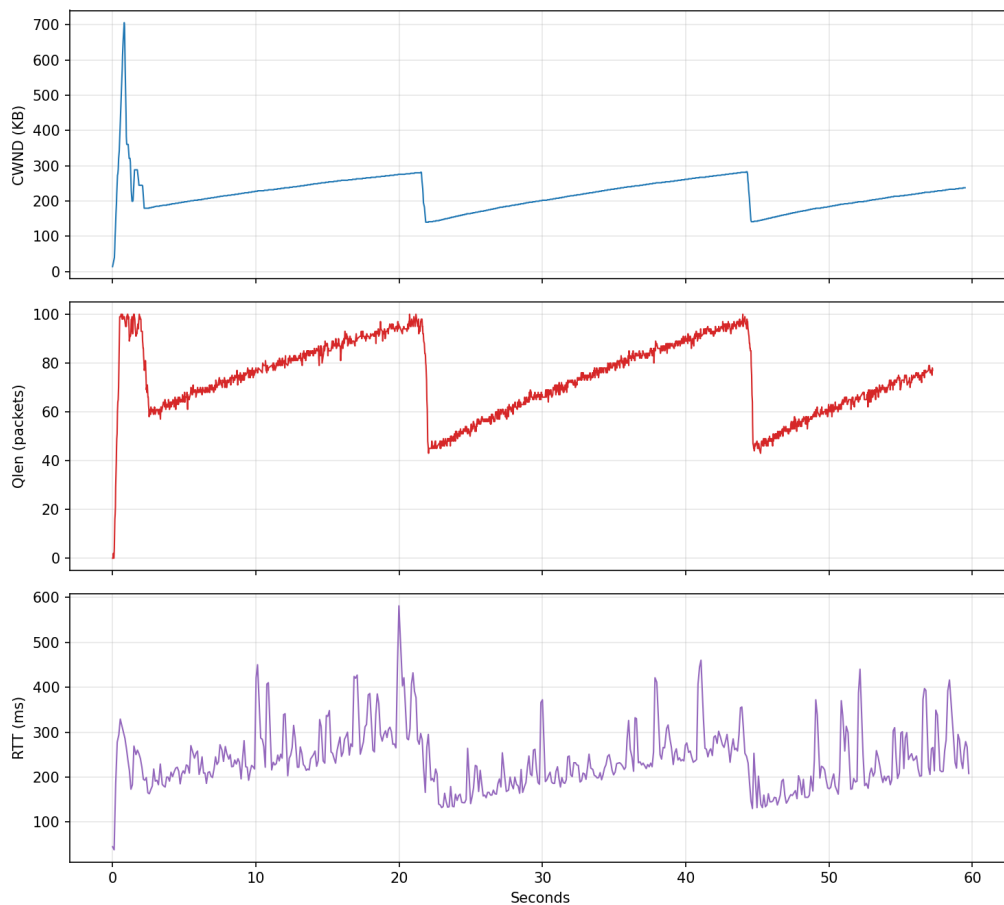


图 2: Bufferbloat 现象重现

可以看到,当 r1-eth1 的队列长度 (qlen) 达到峰值的时候,h1 的拥塞窗口值 (cwnd) 达到了最大值,h1 与 h2 间的往返延迟 (rtt) 也随之增加,且三者的变化趋势基本一致。这表明在网络中出现了 Bufferbloat 现象。

接下来选取 20,100,200 长度的队列进行试验,对 cwnd,qlen,rtt 分别进行进一步的分析。

3.1 cwnd 分析

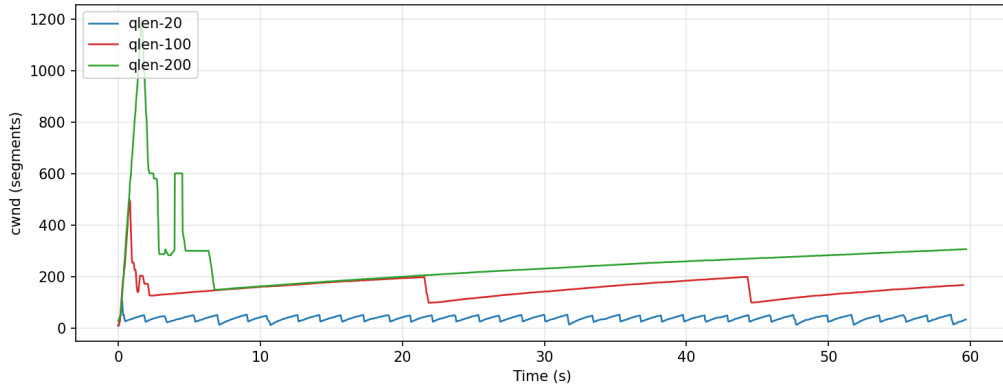


图 3: 不同队列长度下 cwnd 变化对比

拥塞窗口值 (cwnd) 反映了 TCP 连接中允许未确认数据包的最大数量。通过观察上图,可以发现,在实验开始时,三种不同队列长度下的 cwnd 值都迅速上升,分别到达峰值。其中,队列长度越长,到达峰值的时间越晚,峰值也越高,大约为队列长度的 5-6 倍。接着,由于队列拥塞,出现丢包现象,发送方接收到信号,降低发送速率, cwnd 值迅速下降;当网络拥塞缓解后, cwnd 值开始恢复。队列越长,下降的幅度越大,且恢复的时间越长。随后进入一个周期性的波动阶段, cwnd 值在一定范围内上下波动,峰值约为队列长度的两倍,谷值略低于队列长度。

3.2 qlen 分析

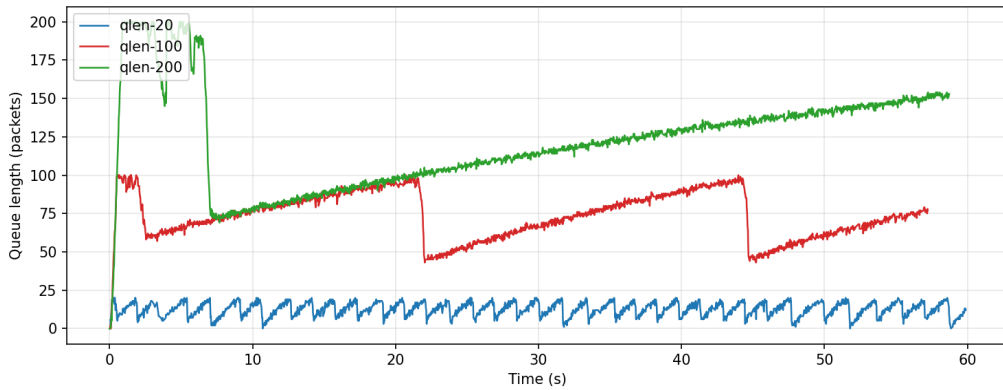


图 4: 不同队列长度下 qlen 变化对比

队列长度 (qlen) 表示路由器或交换机中等待处理的数据包数量。通过观察上图,可以发现,在实验开始时,三种不同队列长度下的 qlen 值都迅速上升,分别到达峰值,峰值大小即为队列长度。随后,由于队列拥塞,出现丢包现象,路由器或交换机开始丢弃数据包, qlen 值迅速下降,大约减少至峰值的一半左右。接着,随着网络拥塞的缓解, qlen 值开始恢复,再次上升至峰值,进入一个新的周期。可以看到,队列长度越长, qlen 值的波动幅度越大,且周期越长。

3.3 rtt 分析

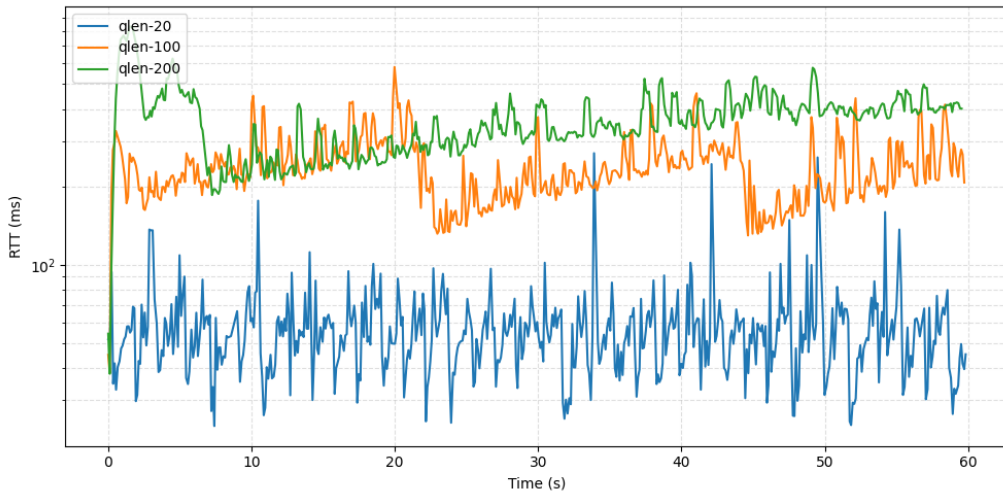


图 5: 不同队列长度下 rtt 变化对比

往返延迟 (rtt) 表示数据包从发送方到接收方再返回发送方所需的时间。通过观察上图, 可以发现, 在实验开始时, 由于队列迅速填满, rtt 值迅速上升, 达到峰值。随后, 由于队列拥塞, 发送速率降低, 队列逐渐排空, rtt 值迅速下降。接着, 随着网络拥塞的缓解, rtt 值开始恢复, 再次上升至峰值, 进入一个新的周期。可以看到, 队列长度越长, rtt 值在整个周期中都普遍高于其他队列长度, 且波动周期越长。这说明, 较长的队列会导致更严重的 bufferbloat 现象, 增加网络延迟。

发现 rtt 图像有很多毛刺, 可能是由于测量误差或者模拟网络环境的影响。

3.4 iperf 吞吐率分析

- 队列长度为 20 时, 平均带宽约为 7.94 Mbits/sec, 峰值带宽 22.6 Mbits/sec, 最低带宽 929 Kbits/sec。
- 队列长度为 100 时, 平均带宽约为 7.40 Mbits/sec, 峰值带宽 46.1 Mbits/sec, 最低带宽 120 Kbits/sec。
- 队列长度为 200 时, 平均带宽约为 7.44 Mbits/sec, 峰值带宽 42.6 Mbits/sec, 最低带宽 85.1 Kbits/sec。

可以看到, 随着队列长度的增加, 平均带宽变化不大, 但峰值带宽有所提升, 而最低带宽则显著下降。这表明较长的队列会导致更严重的 bufferbloat 现象, 影响网络性能。

四、 解决 BufferBloat 问题

利用 RED, CoDel 以及 Tail Drop 三种不同的队列管理算法分别进行测试, 结果如下所示:

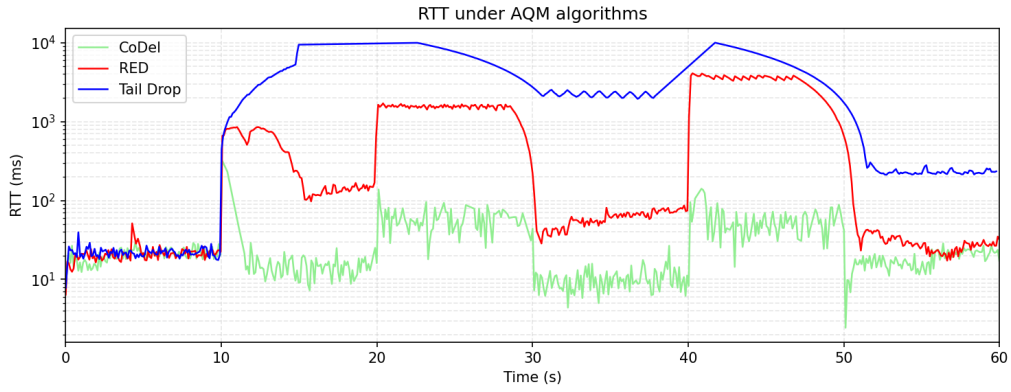


图 6: 不同队列管理算法下的吞吐率变化对比

可以看到,三种不同的队列管理算法的效果优劣顺序为:CoDel > RED > Tail Drop。在缓解 BufferBloat 问题方面,CoDel 表现最佳,能够有效降低网络延迟并提高吞吐率;RED 次之,也能在一定程度上缓解 BufferBloat 问题;而 Tail Drop 效果最差,无法有效解决 BufferBloat 问题。这是因为:

- Tail Drop 是一种简单的队列管理算法,他仅仅在队列满时丢弃数据包,无法主动预防拥塞,因此难以缓解 BufferBloat 问题。
- RED 是一种主动队列管理算法,通过在队列达到一定阈值时随机丢弃数据包,来预防拥塞的发生,从而降低延迟并提高吞吐率。因此,相较于 Tail Drop,RED 能够更有效地缓解 BufferBloat 问题。
- CoDel 是一种先进的主动队列管理算法,通过动态调整丢包概率来控制队列长度,能够更精确地预防拥塞的发生,从而显著降低延迟并提高吞吐率。可以看出,CoDel 在缓解 BufferBloat 问题方面表现最佳。

同时,我们发现图像中三种方法的峰值都会稳定维持一段时间,这是可能是因为 TCP 调整机制有一定的时间延迟,导致在 rtt 迅速上升到达峰值后,TCP 仍然维持较高的发送速率一段时间,直到感知到网络拥塞并调整发送速率,从而使得吞吐率在峰值处维持一段时间。

在复现 BufferBloat 问题的实验中,我们发现 tail drop 算法的图像和标准结果有区别,体现在曲线一直出现“尖端”情况。这是仿真环境的差异导致的。在 Mininet 中,maxq 参数实际上是一个模拟延迟和丢包的批处理大小,与真实的队列大小有区别,只是近似仿真。